

Checkpoint 3.3 — Employee Profile Schema & Client Profile Refinements

Amen Bank — Real-Time Anomaly Detection Platform

Internship Project Documentation

June 3, 2026

1 Purpose

This checkpoint covers two things:

1. Two refinements to the **client profile** (Checkpoint 3.2) that closed real gaps in the design: tracking of known beneficiaries / emitters, and a sequence buffer to support the LSTM model.
2. The complete **employee profile schema** — hot and cold — following the same hot/-cold methodology as the client profile.

2 Client Profile Refinements

2.1 Gap 1 — Beneficiary and Emitter Tracking

Problem identified. The raw event schema (Checkpoint 3) was designed with **beneficiary_account** (virement) and **emitter_account** (cheque) fields, specifically because “first-time beneficiary” is a critical fraud signal. However, the client profile in Checkpoint 3.2 contained no field tracking each client’s history of beneficiaries or emitters. The Enrichment Service had no way to compute “is this beneficiary new for this client?”

Resolution. Two new structures added to Category 3 (Behavioral Patterns) of the client hot profile.

```
known_beneficiaries_365d : {
  beneficiary_account -> {
    first_seen : date,
    count      : int,
    total_amount : float,
    last_seen  : date
  }
}

known_emitters_365d : {
  emitter_account -> {
    first_seen : date,
    count      : int,
    total_amount : float,
    last_seen  : date
  }
}
```

Window choice — 365 days. Chosen as the sweet spot between 180d (misses annual recurring beneficiaries: landlords, insurance, school fees) and lifetime (unbounded growth). A beneficiary unused for over a year is effectively “unknown” again — if it reappears with a large amount, this is itself a legitimate fraud signal.

2.2 Gap 2 — LSTM Sequence Buffer

Problem identified. The Autoencoder consumes a flat feature vector — the existing profile fields feed it directly. However, the **LSTM model consumes a sequence** of past events for the client. The aggregate fields in the hot profile cannot be unrolled back into a sequence. The architecture had no defined source for the LSTM input.

Options considered:

Option	Description	Trade-off
A — Query PostgreSQL	Enrichment Service queries the transactions table for the last N events at scoring time.	Always fresh, no extra storage; but adds DB read latency to the real-time critical path.
B — Redis buffer	Hot profile keeps a bounded sliding window of recent events.	Constant low latency; bounded memory; requires the streaming pipeline to push updates back to Redis.
C — Hybrid (RAM with DB fallback)	Use Redis when available, fall back to PostgreSQL when memory saturates.	Rejected: inconsistent latency, unpredictable capacity, double code path.

Decision: Option B. Add a sixth category to the client hot profile.

```
// Category 6 - Sequence Buffer (LSTM input)
recent_events_buffer : list[event] // bounded, length <= 50
// Maintained via Redis LPUSH + LTRIM
// Each entry: minimal event JSON (op_type, amount, timestamp,
//                branch_id, employee_id, account_id)
```

Why bounded: memory becomes deterministic ($N_{clients} \times 50 \times event_size$), latency is constant, and oldest events drop off automatically. If LSTM requires a longer context, the buffer size becomes a tuning parameter.

2.3 Updated Client Profile Field Count

Category	Fields / structures
1. Personal info	7
2. Transaction statistics	113
3. Behavioral patterns	12 (10 distributions + 2 history maps)
4. Financial state	12
5. Risk history	49
6. Sequence buffer	1 (bounded list)
Total	~194 + sequence buffer

3 Employee Profile — Design Philosophy

The employee is the **processor** of transactions, not the actor with money. This reframes every category compared to the client profile.

3.1 Categories Used

1. **Personal Info** — identity, role, branch assignment, tenure
2. **Transaction Statistics** — same structure as client, but counts *processed* transactions
3. **Behavioral Patterns** — where, when, which clients, which operations they process
4. **Peer Comparison** — new category, unique to employees
5. **Risk History** — alerts involving them

3.2 Categories Dropped

- **Financial state** — intentionally excluded. An employee's personal balance is investigative HR/internal-audit territory, not transaction anomaly detection. Out of scope.

3.3 Employee Cold-Start — Transfer Sensitivity

Domain consideration. In Tunisia, employees can be transferred between branches with little notice. Comparing a freshly transferred employee to their new peers on a 30-day window would unfairly flag them.

Resolution. Mirror the cold-start handling for new clients:

- `last_transfer_date` — tracks the most recent assignment change.
- `maturity_status` — enum [`transitioning`, `established`].
- During `transitioning`, peer comparisons are either skipped or down-weighted; behavioral distributions are interpreted with caution.

4 Employee Hot Profile (Redis)

4.1 Category 1 — Personal Info (6 fields)

<code>employee_id</code>	: string	(primary key)
<code>assigned_branch_id</code>	: string	
<code>role</code>	: enum	[<code>teller</code> , <code>supervisor</code> , ...]
<code>hire_date</code>	: date	
<code>last_transfer_date</code>	: date	
<code>maturity_status</code>	: enum	[<code>transitioning</code> , <code>established</code>]

4.2 Category 2 — Transaction Statistics (113 fields)

Same structure as the client profile, but counts *processed* transactions.

```
days_since_last_processed_transaction : int

stats_<operation_type>_<window> {
  count          : int
  sum_amount     : float
  mean_amount    : float
  std_amount     : float
  rejected_count : int
  reversed_count : int
  near_threshold_ratio : float
}
// 4 ops x 4 windows x 7 fields + 1 = 113
```

4.3 Category 3 — Behavioral Patterns (10 distributions)

```
behavior_30d {
  branch_distribution      : {branch_id      -> frequency}
  hour_distribution        : {hour_bucket    -> frequency}
  dow_distribution         : {day_of_week     -> frequency}
  operation_distribution   : {operation_type  -> frequency}
  client_distribution      : {client_id      -> frequency}
}

behavior_180d {
  // same 5 distributions
}
```

Note: `branch_distribution` is retained (not collapsed to a single `assigned_branch_id`) because employees can have part-time assignments or cover other branches. `employee_distribution` from the client profile is replaced by `client_distribution` here.

4.4 Category 4 — Peer Comparison (10 fields) — NEW

Each metric is computed for this employee and converted to a z-score against peers (same branch, same role).

```
peer_comparison_30d {
  z_volume                : float
  z_error_rate             : float
  z_near_threshold_ratio   : float
  z_client_concentration   : float
  z_client_location_mismatch : float
}

peer_comparison_90d {
  // same 5 z-scores
}
```

Metric definitions:

- `z_volume` — total operations processed.
- `z_error_rate` — rejection + reversal rate.
- `z_near_threshold_ratio` — proportion of operations in 8 000–9 999 DT band (potential smurfing facilitator).
- `z_client_concentration` — how concentrated their clientele is on a few specific clients (potential collusion marker).
- `z_client_location_mismatch` — rate of processing clients whose `home_branch_id` differs from the employee's assigned branch.

4.5 Category 5 — Risk History (49 fields)

Same structure as the client risk history — counts of alerts the employee was involved in, split by tier, window, and supervisor decision.

```
days_since_last_alert      : int

count_<tier>_<window>      : int
```

```
confirmed_count_<tier>_<window> : int
rejected_count_<tier>_<window> : int
// 4 tiers x 4 windows x 3 fields + 1 = 49
```

4.6 Employee Hot Profile — Field Count

Category	Fields / structures
1. Personal info	6
2. Transaction statistics	113
3. Behavioral patterns	10 distributions
4. Peer comparison	10
5. Risk history	49
Total	~188 per employee

5 Cold Store Additions

The employee profile introduces two new tables in PostgreSQL on top of those defined in Checkpoint 3.2.

```
employees_master {
    employee_id (PK), assigned_branch_id, role,
    hire_date, last_transfer_date, status, ...
}

peer_baselines {
    // Aggregated per (branch_id, role) for z-score computation.
    // Recomputed by the batch pipeline.
    branch_id, role, snapshot_date,
    mean_volume_30d, std_volume_30d,
    mean_error_rate_30d, std_error_rate_30d,
    mean_near_threshold_ratio_30d, std_near_threshold_ratio_30d,
    mean_client_concentration_30d, std_client_concentration_30d,
    mean_client_location_mismatch_30d, std_client_location_mismatch_30d,
    // and the same metrics for the 90d window
}
```

The `alerts` and `transactions` tables defined in Checkpoint 3.2 already support employee-side aggregation through their `employee_id` foreign key.

6 Open Questions

1. **Peer set definition.** Should small branches (e.g., fewer than 5 tellers) fall back to a wider peer set (region-level or role-only) for statistically meaningful z-scores?
2. **Peer baseline refresh cadence.** Nightly, weekly, or some other cadence? Affects how quickly the system adapts as employees join, leave, or transfer.
3. **Peer set membership.** Does it include recently transferred employees? Part-time tellers? Define rigorously.
4. **Maturity threshold for employees.** How many days after transfer before `maturity_status` flips from `transitioning` to `established`? Tuning parameter.

5. **Distribution truncation.** As with the client profile, `client_distribution` at 180d can grow unbounded. Strategy for capping (top-N or frequency threshold) needed.

These will be revisited during implementation of the Profile Manager service.

7 Next Step

Design the **enriched event schema** — the output of the Enrichment Service after joining a raw event with both the client and employee profiles. This is the actual feature vector that the Scoring Service consumes.